

Cloud Web Server Security Report

Name: Festus Ochieng' Obong'o

Date: January 25, 2026

Role: Junior DevOps Engineer

Understanding Customer Challenges and Requirements

Client Challenges

- **Current Setup and Issues:** The client currently operates a local web server hosted via **Docker** on a single physical machine. This setup is isolated, lacks automated orchestration, and relies on manual configuration.
 - **Limitations & Challenges:**
 - * **Scalability:** A local-only setup cannot handle sudden traffic spikes without manual hardware upgrades.
 - **Compliance:** Physical access is harder to audit, and local environments often lack the standardized encryption and logging required by **GDPR** or **HIPAA**.
 - **Disaster Recovery:** A single hardware failure or local power outage results in total downtime, as there are no geographically redundant backups or failover mechanisms.
-

Audit Results

Baseline Security Posture

- **Hardening Index Score:** 54
 - **Identified Vulnerabilities:**
 - **AUTH-9208:** Duplicate UIDs (Risk of privilege escalation).
 - **FILE-6310:** World-writable files (Unauthorized data modification).
 - **PKGS-7392:** 95 outdated packages (Known CVE exploits).
 - **NETW-2705:** Missing HTTPS/TLS (Data intercepted in transit).
 - **Connection to Business Challenges:** Missing HTTPS directly violates compliance standards, exposing the company to legal fines. Outdated packages create "backdoors" that could be exploited to delete data, making disaster recovery impossible if the backups are also compromised.
-

Evaluating Cloud Service Models

Service Model Comparison

- **IaaS (Infrastructure as Code):** The client is responsible for the **OS, Middleware, and Apps**. Vulnerabilities like file permissions and hashing rounds remain the client's burden.
- **PaaS (Platform as a Service):** The provider manages the OS and Runtime. Vulnerabilities like outdated system packages are handled by the provider, but the client remains responsible for **Application-level security** and user access.
- **SaaS (Software as a Service):** The provider handles almost everything. The client is only responsible for **Data and User Access**.

Recommended Model

- **Chosen Service Model: PaaS (e.g., AWS Elastic Beanstalk or Azure App Service).**
 - **Justification:** This model removes the burden of patching 95+ OS packages (addressing our audit findings) while allowing the client to focus on secure code. It provides built-in scalability and easier compliance through managed TLS certificates.
-

Exploring Deployment Options

Deployment Choices

- **Public Cloud:** Highly scalable and cost-effective but requires stricter security (MFA, encryption) as resources share physical hardware.
- **Private Cloud:** Maximum security and control, but expensive and harder to scale.
- **Hybrid Cloud:** Offers the best of both worlds—sensitive data stays on-premise while web traffic scales in the public cloud.

Recommended Deployment Option

- **Chosen Deployment Option: Public Cloud.**
 - **Justification:** For an e-commerce or standard web server, the Public Cloud's native **Disaster Recovery** (multi-region availability zones) far outweighs the cost of building a private site.
-

Developing a Cloud Solution Recommendation

Hardening Actions

The Issue	The Action	The Result
Duplicate UIDs	<code>userdel -r [user]</code>	Prevents unauthorized account impersonation.
Outdated Packages	<code>sudo apt update && sudo apt upgrade</code>	Closes known security holes (CVEs).
Weak Hashing	Edited <code>/etc/login.defs</code> to 100k rounds	Protects passwords against brute-force attacks.

Cloud Solution Recommendation

I recommend migrating to a **Public Cloud PaaS** environment. This integrates the hardening lessons by using **Identity and Access Management (IAM)** to replace local user management and using **Managed Load Balancers** to enforce HTTPS automatically.

Disaster Recovery Planning

Proposed Recovery Plan

- **Backups:** Implement **Automated Snapshots** of the database and **Version Control (Git)** for server configurations. Regular backups ensure a Recovery Point Objective (RPO) of less than 1 hour.
 - **Failover Mechanisms:** Use **Multi-AZ (Availability Zone)** deployment. If one data center fails, the load balancer shifts traffic to a healthy instance in another city.
 - **Monitoring and Alerts:** Use tools like **Prometheus** or **CloudWatch**. Proactive alerts on 4xx/5xx errors allow engineers to fix issues before they lead to total system failure.
-

Reflection on the Cloud Process

- **Understanding Challenges:** Analyzing the "local-only" setup highlighted that security isn't just about locks; it's about availability and uptime.
- **Evaluating Service Models:** I learned that the **Shared Responsibility Model** is key. You can't secure what you don't manage, and you must know what you are responsible for.

- **Exploring Deployment Options:** I realized that "Public Cloud" doesn't mean "unsecure"—it just means the security focus shifts to identity and encryption.
- **Hardening and Recommendations:** The hardening process proved that manual fixes are tedious. Moving to the cloud allows us to automate these fixes at scale.
- **Disaster Recovery:** I now see DR as the "final boss" of security. All the hardening in the world doesn't matter if the server room floods and you have no backup.

Overall Reflection

This lab bridged the gap between **technical implementation** (changing UUIDs/permissions) and **strategic decision-making** (choosing a cloud model). The key takeaway is that security is a continuous lifecycle. In the real world, I will apply these skills by prioritizing vulnerabilities that impact business continuity and using automation to ensure that "hardened" states don't drift over time.